

PHASE 03 / 07

Querying Basics & Execution Flow

Ask your data precise questions with SELECT, WHERE, GROUP BY, HAVING and ORDER BY, and learn the hidden order in which MySQL actually evaluates them.

SQL Basic Commands | SQL Execution Flow

Most SQL bugs are not typos; they come from a wrong mental model of what happens first. This phase gives you the right model, backed by real row counts from your own database.

CONTENTS

Inside this phase

SQL Basic Commands 3

- Anatomy of a SELECT 3
- SELECT, DISTINCT and LIMIT 4
- WHERE: filtering rows 6
- LIKE: searching text patterns 8
- ORDER BY: sorting and Top-N 9
- Aliases: giving things better names 10
- Aggregate functions: many rows in, one value out 10
- GROUP BY: aggregates per bucket 11
- HAVING: filtering the buckets 13

SQL Execution Flow 14

- Written order vs logical order 14
- Watching the rows shrink: a real example 15
- What the logical order explains 16
- Writing faster, safer queries 17
- Recipe: profile a new table in five queries 17

Phase 3 Recap & Practice 19

- Cheat sheet 19
- Check yourself 19
- Practice lab 20
- Answer key 20

3.1 PART 3.1 · THE SELECT TOOLKIT

SQL Basic Commands

SELECT is the command you will write more than all others combined. It reads data without changing it, so it is safe to experiment with. This chapter walks through every clause from your course, one at a time.

What you will learn

- Read the anatomy of a `SELECT` statement
- Filter rows with `WHERE`, `AND/OR`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`
- Sort and limit results; page through data
- Summarize data with aggregate functions
- Group rows with `GROUP BY` and filter groups with `HAVING`

Why it matters for data engineering. Analysts ask questions in business language; you translate them into WHERE, GROUP BY and HAVING. The same clauses appear in every warehouse and in dbt models, so this vocabulary is permanent.

Anatomy of a SELECT

A query is built from **clauses**, each with one job. Only `SELECT` and (usually) `FROM` are required; add the others as the question demands. Written order is fixed, so memorize it:



Figure 1. One question, seven clauses. Each clause is optional except SELECT, and they must be written in exactly this order.

SQL — The query from the diagram, executed

```sql
SELECT   category, ROUND(AVG(unit_price), 2) AS avg_price
FROM     dim_product
WHERE    unit_price > 100
GROUP BY category
HAVING   AVG(unit_price) > 500
ORDER BY avg_price DESC
LIMIT    3;
```

RESULT — 3 ROWS

category	avg_price
Clothing	584.17
Toys	564.37
Electronics	545.42

SELECT, DISTINCT and LIMIT

SQL — Course ch-3: SELECT * returns every column

```sql
SELECT * FROM dim_customer LIMIT 3;
```

RESULT — 3 ROWS

customer_key	customer_id	first_name	last_name	gender	email	phone	country	state	city
1	CUST0001	Antonio	Kelly	M	richard16@examp…	9904005277	Tunisia	New Hampshire	Lake Timothy
2	CUST0002	Todd	Lucas	F	danielsnyder@ex…	0012907800627	Djibouti	Wyoming	Nicholasstad
3	CUST0003	Matthew	Lee	M	joseph40@exampl…	001596505299143	Guinea-Bissau	Montana	Janiceton

SQL — Name only the columns you need (the course uses LIMIT 15)

```sql
SELECT customer_id, email FROM dim_customer LIMIT 5;
```

RESULT — 5 ROWS

customer_id	email
CUST0001	richard16@example.net
CUST0002	danielsnyder@example.org
CUST0003	joseph40@example.net
CUST0004	juan09@example.net
CUST0005	wroberts@example.net

SQL — DISTINCT removes duplicate rows from the result

```sql
SELECT DISTINCT category FROM dim_product ORDER BY category;
```

RESULT — 6 ROWS

category
Books
Clothing
Electronics
Home & Kitchen
Sports
Toys

SQL — A column can be a calculation (more in Phase 5)

```sql
SELECT product_name, unit_price, unit_price * 0.90 AS discounted FROM dim_product ORDER BY
product_key LIMIT 3;
```

RESULT — 3 ROWS

product_name	unit_price	discounted
Figure Method	897.35	807.6150
Pressure That	920.58	828.5220
Huge Change	553.56	498.2040

`LIMIT n` keeps the first n rows. To page through results, add an offset:

SQL — Pagination

```sql
SELECT product_key, product_name FROM dim_product ORDER BY product_key
LIMIT 10 OFFSET 20;      -- skip 20 rows, return the next 10  (page 3 of 10-row pages)
-- MySQL also accepts the shorthand  LIMIT 20, 10   (offset FIRST, then count!)
```

⚠ WATCH OUT

`LIMIT` without `ORDER BY` returns **arbitrary** rows; the database is free to pick any, and the choice may change between runs. Always sort when the *which* rows matters.

DATA-ENGINEERING LENS

Treat `SELECT *` as an exploration tool only. In pipelines, name every column: when someone adds a column upstream, `SELECT *` silently changes your output schema and can break downstream loads.

WHERE: filtering rows

`WHERE` keeps only the rows for which a condition is TRUE. Everything else is discarded *before* any grouping.

Operator	Meaning	Example
`=` `<>` (or `!=`)	equal / not equal	`gender = 'F'`
`<` `<=` `>` `>=`	comparisons (dates and text work too)	`join_date > '2022-01-01'`
`BETWEEN a AND b`	inclusive range	`unit_price BETWEEN 100 AND 200`
`IN (…)`	matches any value in a list	`country IN ('France','Spain')`
`LIKE`	text pattern (`%` any length, `_` one char)	`first_name LIKE 'T%'`
`IS NULL` / `IS NOT NULL`	missing-value tests (never use `= NULL`)	`phone IS NULL`
`AND` `OR` `NOT`	combine conditions	`a AND (b OR c)`

SQL — Course ch-3 (1): a single condition

```sql
SELECT customer_id, first_name, gender, country FROM dim_customer WHERE gender = 'F' ORDER BY
customer_key LIMIT 4;
```

RESULT — 4 ROWS

customer_id	first_name	gender	country
CUST0002	Todd	F	Djibouti
CUST0004	Gregory	F	Tajikistan
CUST0005	Nathan	F	Tokelau
CUST0009	Shawn	F	Guatemala

AND / OR and the parentheses trap

`AND` is evaluated **before** `OR`, exactly like multiplication before addition. The course example uses parentheses to say: *female customers who are either from France or joined after 2022.* Watch what happens without them.

SQL — With parentheses (the intent)

```sql
SELECT COUNT(*) AS customers
FROM dim_customer
WHERE gender = 'F'
  AND (country = 'France'
       OR join_date > '2022-01-01');
```

RESULT — 1 ROW

customers
199

SQL — Without them: AND binds first

```sql
SELECT COUNT(*) AS customers
FROM dim_customer
WHERE gender = 'F'
  AND country = 'France'
  OR  join_date > '2022-01-01';
```

RESULT — 1 ROW

customers
388

How MySQL reads the unparenthesized version:

`gender='F' AND country='France'` | `OR` | `join_date > '2022-01-01'`

→ includes EVERY customer who joined after 2022, regardless of gender or country: 388 rows instead of 199.

When mixing AND with OR, always add parentheses so the next reader (and the optimizer) cannot misread you.

Figure 2. Precedence: NOT, then AND, then OR.

BETWEEN, IN and the NOT IN trap

SQL — BETWEEN includes both ends (990 ≤ price ≤ 1000)

```sql
SELECT product_name, unit_price FROM dim_product WHERE unit_price BETWEEN 990 AND 1000 ORDER BY unit_price;
```

RESULT — 1 ROW

product_name	unit_price
Likely Brother	999.04

SQL — IN is shorthand for several ORs

```sql
SELECT DISTINCT category FROM dim_product WHERE category IN ('Books', 'Toys') ORDER BY category;
```

RESULT — 2 ROWS

category
Books
Toys

SQL — One NULL inside NOT IN poisons the whole test

```sql
SELECT 5 NOT IN (1, 2, 3) AS clean_list, 5 NOT IN (1, 2, NULL) AS list_with_null;
```

RESULT — 1 ROW

clean_list	list_with_null
1	*NULL*

⚠ WATCH OUT

`x NOT IN (1, 2, NULL)` is **never true**: SQL cannot be sure `x` differs from an unknown value, so it answers NULL, and `WHERE` discards NULL. If the list comes from a subquery that might contain NULLs, use `NOT EXISTS` or filter the NULLs out (Phase 6).

NULL and three-valued logic

A comparison involving NULL gives **UNKNOWN**, not TRUE or FALSE. `WHERE` keeps only TRUE.

A	B	A AND B	A OR B	NOT A
TRUE	NULL	NULL	TRUE	FALSE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL
NULL = NULL		NULL (not TRUE!)		

SQL — NULL is contagious in arithmetic and comparisons

```sql
SELECT NULL = NULL AS equals, NULL IS NULL AS is_null, 1 + NULL AS arithmetic;
```

RESULT — 1 ROW

equals	is_null	arithmetic
NULL	1	*NULL*

LIKE: searching text patterns

`%` stands for *any number of characters (including none)*; `_` stands for *exactly one character*. The course tests both patterns:

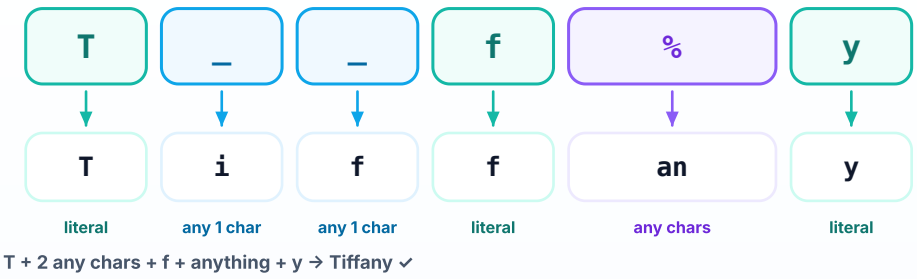


Figure 3. How each part of the pattern consumes characters of the value.

SQL — Course ch-3 LIKE (1)

```sql
SELECT COUNT(*) AS names_starting_with_T FROM dim_customer WHERE first_name LIKE 'T%';
```

RESULT — 1 ROW

names_starting_with_T
33

SQL — Course ch-3 LIKE (2): the pattern from the diagram

```sql
SELECT customer_key, first_name, last_name FROM dim_customer WHERE first_name LIKE 'T__f%y' ORDER BY customer_key;
```

RESULT — 2 ROWS

customer_key	first_name	last_name
34	Tiffany	Gilmore
91	Tiffany	Reed

- `LIKE 'a%'` starts with a · `'%a'` ends with a · `'%a%'` contains a.
- A leading wildcard (`'%a'`) cannot use an index, so it scans the whole table. Prefer `'a%'` when you can.
- For regular expressions, use `REGEXP`: `email REGEXP '^[a-z]+[0-9]+@'`. Text comparisons are case-insensitive by default collations in MySQL.

ORDER BY: sorting and Top-N

SQL — Course ch-3 'Sorting': the 3 most expensive products

```sql
SELECT product_name, unit_price FROM dim_product ORDER BY unit_price DESC LIMIT 3;
```

RESULT — 3 ROWS

product_name	unit_price
Likely Brother	999.04
Soon During	989.41
Car Report	986.45

SQL — Several sort keys: category A→Z, then price high→low within each

```sql
SELECT category, brand, unit_price FROM dim_product ORDER BY category ASC, unit_price DESC LIMIT 4;
```

RESULT — 4 ROWS

category	brand	unit_price
Books	BrandB	978.88
Books	BrandD	975.35
Books	BrandB	962.74
Books	BrandA	941.43

- `ASC` (default) smallest first · `DESC` largest first. NULLs sort **first** in `ASC` in MySQL.
- You may sort by an alias or by position (`ORDER BY 2 DESC`). Positions are handy when exploring but fragile in saved code.
- `ORDER BY … LIMIT n` is the classic **Top-N** pattern; the optimizer can often do it without sorting everything.

Aliases: giving things better names

An alias (`AS`) renames a column or table *in the output only*; nothing changes in storage. Your course shows a quoted alias with a space:

SQL — Course ch-3 'ALIAS'

```sql
SELECT product_key, product_id, product_name AS 'product name', category FROM dim_product LIMIT 3;
```

RESULT — 3 ROWS

product_key	product_id	product name	category
1	PROD0001	Figure Method	Clothing
2	PROD0002	Pressure That	Clothing
3	PROD0003	Huge Change	Clothing

PRO TIP

Prefer **snake_case aliases without spaces** (`product_name`, `avg_price`). Names with spaces need quotes everywhere they are used and break many BI tools. If you must, wrap them in backticks: `` `product name` ``.

Aggregate functions: many rows in, one value out

Function	Returns	Ignores NULL?
`COUNT(*)`	number of rows	No: counts every row
`COUNT(col)`	number of rows where col is not NULL	Yes
`COUNT(DISTINCT col)`	number of different non-NULL values	Yes
`SUM(col)` · `AVG(col)`	total / mean	Yes (AVG divides by non-NULL count)
`MIN(col)` · `MAX(col)`	smallest / largest (numbers, dates, text)	Yes

SQL — Scores are 10, NULL, 30, NULL

```sql
SELECT COUNT(*)     AS all_rows,
       COUNT(score) AS non_null,
       SUM(score)   AS total,
       AVG(score)   AS average
FROM   nulls_demo;
```

RESULT — 1 ROW

all_rows	non_null	total	average
4	2	40	20.0000

REMEMBER

`AVG(score)` is **20**, not 10: NULLs are skipped, so it averages only 10 and 30. If missing should count as zero, say so explicitly: `AVG(COALESCE(score, 0))`.

SQL — An aggregate without GROUP BY summarizes the whole table into one row

```sql
SELECT COUNT(*)                 AS customers,
       COUNT(DISTINCT country)  AS countries,
       MIN(join_date)           AS first_join,
       MAX(join_date)           AS last_join
FROM   dim_customer;
```

RESULT — 1 ROW

customers	countries	first_join	last_join
500	207	2020-10-12	2025-10-11

GROUP BY: aggregates per bucket

Without `GROUP BY`, an aggregate collapses the *whole table* into one row. With it, MySQL first splits the rows into **buckets** (one per distinct value) and aggregates each bucket separately.

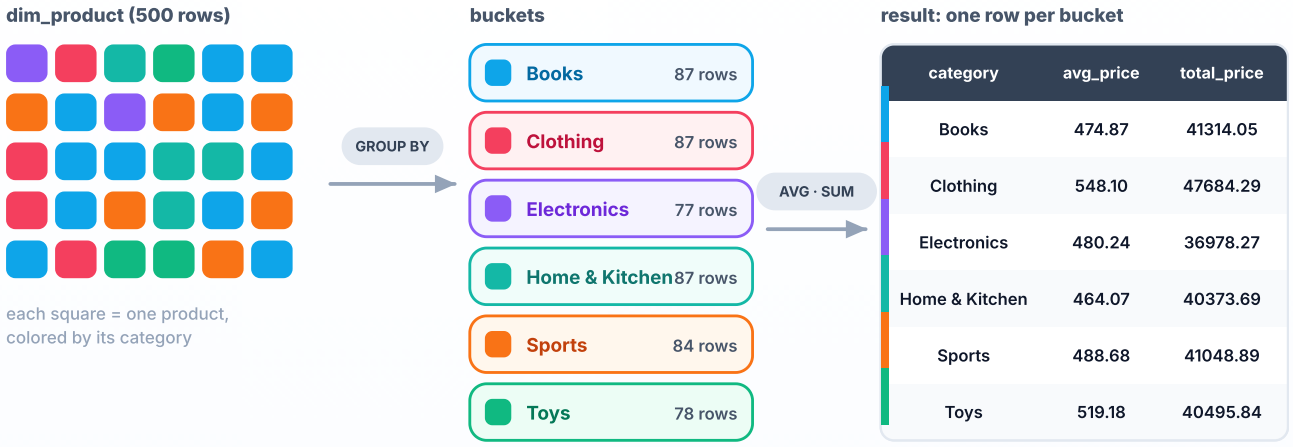


Figure 4. GROUP BY category turns 500 product rows into 6 summary rows. The numbers on the right are real results from your database.

SQL — Course ch-3 'GROUPING' (1)

```sql
SELECT   category,
         AVG(unit_price) AS avg_price,
         SUM(unit_price) AS total_price
FROM     dim_product
GROUP BY category
ORDER BY category;
```

RESULT — 6 ROWS

category	avg_price	total_price
Books	474.874138	41314.05
Clothing	548.095287	47684.29
Electronics	480.237273	36978.27
Home & Kitchen	464.065402	40373.69
Sports	488.677262	41048.89
Toys	519.177436	40495.84

SQL — Group by two columns: one bucket per (category, brand) pair

```sql
SELECT   category, brand, COUNT(*) AS products
FROM     dim_product
GROUP BY category, brand
ORDER BY category, brand
LIMIT 6;
```

RESULT — 6 ROWS · 30 BUCKETS IN TOTAL

category	brand	products
Books	BrandA	17
Books	BrandB	19
Books	BrandC	19
Books	BrandD	16
Books	BrandE	16
Clothing	BrandA	17

SQL — The golden rule: every selected column must be grouped or aggregated

```sql
SELECT category, product_name, AVG(unit_price) FROM dim_product GROUP BY category;
```

ERROR

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'sales.dim_product.product_name' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

REMEMBER

Golden rule of GROUP BY: each column in `SELECT` must either appear in `GROUP BY` or be wrapped in an aggregate. Otherwise MySQL cannot know *which* product_name to show for a bucket of 87 products.

HAVING: filtering the buckets

`WHERE` filters rows *before* grouping, so it cannot see aggregate results. `HAVING` filters buckets *after* grouping, so it can.

SQL — Course ch-3 'GROUPING' (2): only categories averaging above 500

```sql
SELECT   category,
         AVG(unit_price) AS avg_price,
         SUM(unit_price) AS total_price
FROM     dim_product
GROUP BY category
HAVING   avg_price > 500
ORDER BY avg_price DESC;
```

RESULT — 2 ROWS

category	avg_price	total_price
Clothing	548.095287	47684.29
Toys	519.177436	40495.84

	WHERE	HAVING
Filters	individual **rows**	whole **groups**
Runs	before `GROUP BY`	after `GROUP BY`
Can use aggregates (`AVG`, `SUM` ...)?	No	Yes
Speed	Reduces data early: cheap	Works on already-grouped data
Rule of thumb	Use it for every condition you can	Use it only for conditions on aggregates

MYSQL SPECIFIC

Your course uses the alias `avg_price` inside `HAVING`. That is a **MySQL extension**; standard SQL (and PostgreSQL, SQL Server) requires repeating the expression: `HAVING AVG(unit_price) > 500`. Write the full expression if your query must be portable.

3.2 PART 3.2 · HOW A QUERY REALLY RUNS

SQL Execution Flow

You *write* SELECT first, but the database does **not** run it first. SQL has a written order and a different logical order. Once you know the logical order, errors like “unknown column” and “invalid use of group function” stop being mysterious.

What you will learn

- Recite the logical order of query evaluation
- Explain why aliases fail in WHERE but work in ORDER BY
- Explain why aggregates and window functions cannot appear in WHERE
- Follow row counts as they shrink through each step
- Apply a data-profiling recipe to any new table

Why it matters for data engineering. Query tuning, debugging data-quality issues and writing correct transformations all depend on knowing which step sees what. Interviewers love this topic because it separates people who memorized syntax from people who understand the engine.

Written order vs logical order

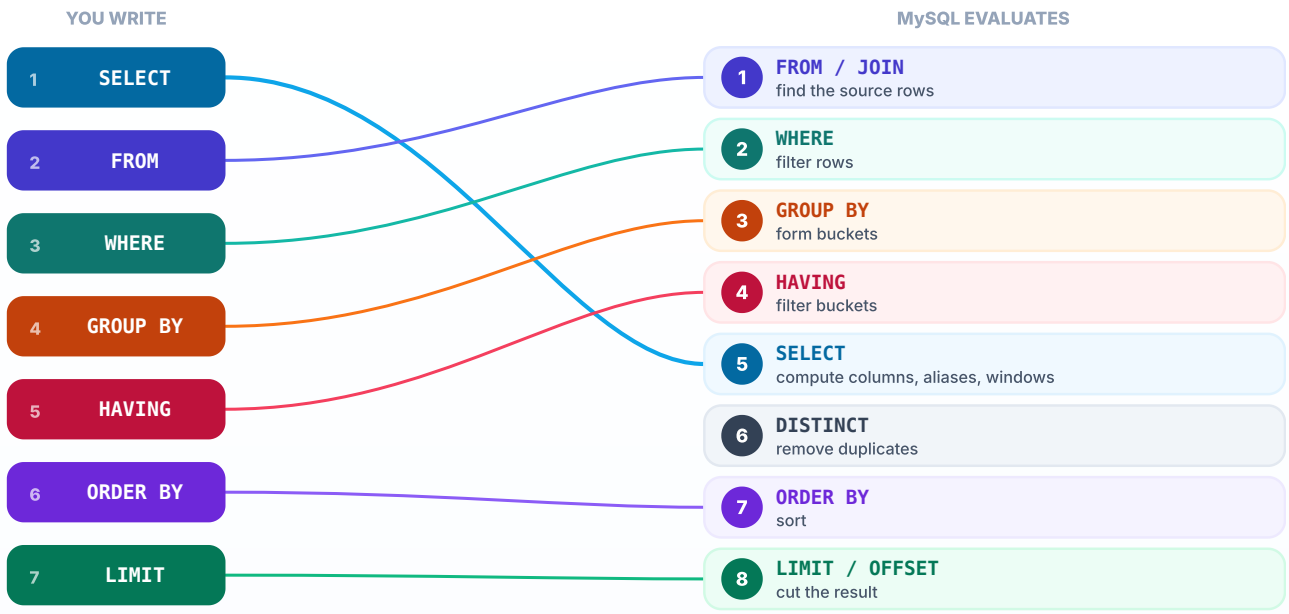


Figure 5. SELECT is written first but evaluated fifth. FROM is written second but evaluated first. Lines show where each written clause lands in the logical order.

REMEMBER

F-W-G-H-S-D-O-L: *From, Where, Group by, Having, Select, Distinct, Order by, Limit.* (In reality `JOIN`s are part of the `FROM` step, and window functions are computed inside `SELECT`, step 5.)

Watching the rows shrink: a real example

Here is a query on `fact_sales`. Below it, the row count after each step, measured by actually running the intermediate queries on your data.

SQL — Top stores by revenue, counting only sales of 3+ units

```sql
SELECT   store_key, ROUND(SUM(total_amount), 2) AS revenue, COUNT(*) AS orders
FROM     fact_sales
WHERE    quantity_sold >= 3
GROUP BY store_key
HAVING   SUM(total_amount) > 300000
ORDER BY revenue DESC
LIMIT    5;
```

RESULT — 5 ROWS

store_key	revenue	orders
19	363685.22	102
17	343583.84	103
21	341325.77	89
37	332382.64	81
22	331082.05	94

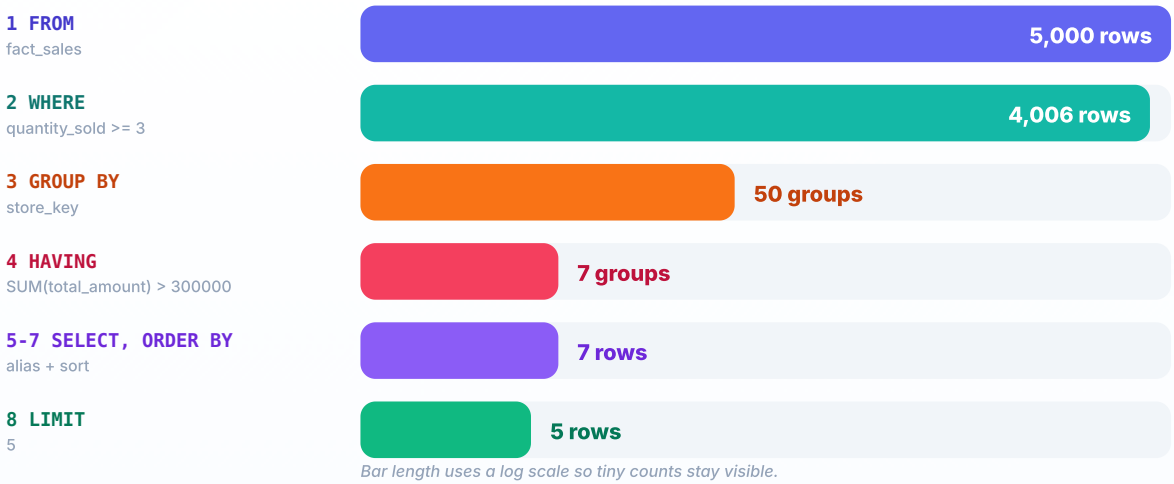


Figure 6. 5,000 rows → 4,006 after WHERE → 50 store buckets → 7 buckets that survive HAVING → top 5 returned.

What the logical order explains

1. Aliases don't exist yet when WHERE runs

SQL — WHERE (step 2) runs long before SELECT (step 5) creates the alias

```sql
SELECT store_key, SUM(total_amount) AS revenue FROM fact_sales WHERE revenue > 1000 GROUP BY
store_key;
```

ERROR

```
ERROR 1054 (42S22): Unknown column 'revenue' in 'where clause'
```

2. Aggregates can't be used in WHERE

SQL — Groups don't exist at step 2. Use HAVING (step 4)

```sql
SELECT store_key FROM fact_sales WHERE SUM(total_amount) > 300000 GROUP BY store_key;
```

ERROR

```
ERROR 1111 (HY000): Invalid use of group function
```

3. ORDER BY can use aliases and aggregates

`ORDER BY` runs at step 7, after `SELECT` has produced the aliases, so `ORDER BY revenue DESC` works (as in the query above). MySQL also lets `GROUP BY` and `HAVING` see aliases as a convenience, but this is non-standard.

4. Window functions can't be filtered in WHERE

Window functions (Phase 6) are computed inside `SELECT` (step 5). To filter on a rank you must compute it in a subquery or CTE, then filter in the outer query. This pattern powers the Top-N-per-group scenario in Phase 7.

You want to...	...in this clause	Because
filter raw rows	`WHERE`	runs 2nd, sees original columns only
filter on `SUM`, `COUNT`, `AVG`	`HAVING`	runs 4th, after groups exist
reuse a computed name	`ORDER BY` (or an outer query / CTE)	aliases are born in step 5
filter on `ROW_NUMBER()`, `RANK()`	outer `WHERE` on a subquery/CTE	windows are computed in step 5
keep only the first n	`LIMIT`	runs last

NOTE

This is the **logical** order: the result must be *as if* the steps ran this way. The optimizer is free to reorder the *physical* work (use an index, push filters down, skip a sort) as long as the answer is identical. `EXPLAIN` shows you the physical plan.

Writing faster, safer queries

Filter early

Put every row-level condition in `WHERE`. Fewer rows entering `GROUP BY` and `JOIN` means less work everywhere.

Select only what you need

Naming columns instead of `*` reduces data movement and protects against schema changes.

Sort last, and only if needed

`ORDER BY` on millions of rows is costly. Skip it in intermediate steps; use it on final output only.

DATA-ENGINEERING LENS

In columnar warehouses (BigQuery, Snowflake) you pay per byte scanned. The habits above (select only needed columns, filter on partition/date columns first) are literally cost control.

Recipe: profile a new table in five queries

Before building anything on a table you have never seen, interrogate it. These queries use only what you learned in this phase, and they surface most data-quality surprises.

SQL — 1 · Size and key uniqueness

```sql
SELECT COUNT(*) AS row_count, COUNT(DISTINCT customer_key) AS distinct_keys FROM dim_customer;
```

RESULT — 1 ROW

row_count	distinct_keys
500	500

SQL — 2 · NULL counts per column (a boolean is 1 or 0, so SUM counts the TRUEs)

```sql
SELECT SUM(email IS NULL)  AS null_email,
       SUM(phone IS NULL)  AS null_phone,
       SUM(city  IS NULL)  AS null_city
FROM   dim_customer;
```

RESULT — 1 ROW

null_email	null_phone	null_city
0	0	0

SQL — 3 · Range check: a negative minimum is suspicious

```sql
SELECT MIN(total_amount) AS min_amount, MAX(total_amount) AS max_amount, ROUND(AVG(total_amount), 2)
AS avg_amount FROM fact_sales;
```

RESULT — 1 ROW

min_amount	max_amount	avg_amount
-27.47	9949.60	2791.49

SQL — 4 · Zoom in on the oddity

```sql
SELECT COUNT(*) AS negative_sales FROM fact_sales WHERE total_amount < 0;
```

RESULT — 1 ROW

negative_sales
6

SQL — 5 · Distribution of a categorical column

```sql
SELECT country, COUNT(*) AS customers FROM dim_customer GROUP BY country ORDER BY customers DESC, country LIMIT 5;
```

RESULT — 5 ROWS

country	customers
Korea	8
Fiji	7
Bolivia	6
Micronesia	6
Zambia	6

DATA-ENGINEERING LENS

The profile found something real: **6 sales have a negative** `total_amount` (a discount larger than the line value). In production you would trace them upstream, then decide: fix, flag or quarantine. Discovering the problem is a `SELECT` away; that is why profiling comes first.

INTERVIEW ANGLE

“In what order does a SELECT execute?” (FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.) “Why can't you use a column alias in WHERE?” (Because WHERE runs before SELECT creates it.) “WHERE vs HAVING?” (Rows before grouping vs groups after.)

3.3 WRAP-UP

Phase 3 Recap & Practice

Consolidate the clauses, then answer six business questions with real queries.

Cheat sheet

Need	Pattern
Pick columns / rows	`SELECT a, b FROM t WHERE cond`
Remove duplicates	`SELECT DISTINCT a FROM t`
Top-N	`ORDER BY x DESC LIMIT n` · paging: `LIMIT n OFFSET m`
Ranges / lists / patterns	`BETWEEN a AND b` · `IN (…)` · `LIKE 'T%'` (`%` many, `_` one)
Missing values	`IS NULL` / `IS NOT NULL`; `NULL = NULL` is not true
Aggregates	`COUNT(*)`, `COUNT(col)`, `COUNT(DISTINCT col)`, `SUM`, `AVG`, `MIN`, `MAX`
Per-group result	`GROUP BY` (every non-aggregated SELECT column must be listed)
Filter groups	`HAVING` · rows use `WHERE`
Evaluation order	FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

Check yourself

Q1 Why does `WHERE gender = 'F' AND a OR b` return unexpected rows?

AND is evaluated before OR, so it reads as (gender='F' AND a) OR b. Add parentheses.

Q2 What is the difference between `COUNT(*)` and `COUNT(col)`?

COUNT(*) counts rows; COUNT(col) counts rows where col is not NULL.

Q3 Can you use a SELECT alias in WHERE? In ORDER BY?

No in WHERE (runs before SELECT); yes in ORDER BY (runs after).

Q4 Why does `x NOT IN (1, NULL)` return no rows?

Comparison with NULL is UNKNOWN, so the whole condition is never TRUE.

Q5 Which clause would you use to keep only categories with more than 80 products?

`GROUP BY category HAVING COUNT(*) > 80`.

Practice lab

Use the `sales` database. Try each question before looking at the answers.

1. List the 5 cheapest products with their names and prices.
2. How many customers are from France, and how many joined in 2023?
3. Show each product `category` with its number of products and highest price, most expensive category first.
4. Which `brand`s have more than 100 products? (`dim_product`)
5. Find all customers whose email ends with `.org` and whose last name starts with `S`; show the first 5 by `customer_key`.
6. In `fact_sales`, how many sales have a quantity of 5 or more *and* a discount above 40?

Answer key

SQL 1

```sql
SELECT product_name, unit_price FROM dim_product ORDER BY unit_price ASC LIMIT 5;
```

RESULT 5 ROWS

product_name	unit_price
Middle Forward	11.82
Look Interest	12.15
Environment Performance	12.22
Us Air	13.22
My Hard	14.24

SQL 2 · conditional counting with boolean sums (a preview of CASE in Phase 5)

```sql
SELECT SUM(country = 'France')          AS from_france,
       SUM(YEAR(join_date) = 2023)      AS joined_2023
FROM   dim_customer;
```

RESULT 1 ROW

from_france	joined_2023
5	101

SQL 3

```sql
SELECT   category, COUNT(*) AS products, MAX(unit_price) AS highest_price
FROM     dim_product
GROUP BY category
ORDER BY highest_price DESC;
```

RESULT 6 ROWS

category	products	highest_price
Toys	78	999.04
Electronics	77	989.41
Sports	84	986.45
Clothing	87	986.00
Home & Kitchen	87	980.27
Books	87	978.88

SQL 4

```sql
SELECT   brand, COUNT(*) AS products
FROM     dim_product
GROUP BY brand
HAVING   COUNT(*) > 100
ORDER BY brand;
```

RESULT 3 ROWS

brand	products
BrandA	112
BrandD	102
BrandE	103

SQL 5

```sql
SELECT customer_key, first_name, last_name, email
FROM   dim_customer
WHERE  email LIKE '%.org' AND last_name LIKE 'S%'
ORDER  BY customer_key
LIMIT  5;
```

RESULT 5 ROWS

customer_key	first_name	last_name	email
79	Mark	Schmidt	jose64@example.org
148	Joshua	Steele	lisa94@example.org
158	Audrey	Soto	bradleyjoe@example.org
205	Daniel	Smith	brian12@example.org
237	Hunter	Smith	sarahmerritt@example.org

SQL 6

```sql
SELECT COUNT(*) AS matching_sales FROM fact_sales WHERE quantity_sold >= 5 AND discount > 40;
```

RESULT 1 ROW

matching_sales
591

COMING IN PHASE 4

Phase 4: Joins & DML. Real questions need several tables at once. You will combine them with INNER, LEFT, RIGHT and FULL joins, then learn to INSERT, UPDATE and DELETE data safely.